

Forge — before/after on one real change

This isn't a mockup. `forge-check.js` is a real pattern-matcher that reads `before.js` and detects these issues by actually parsing the code — nothing here is scripted output.

What went in (AI-generated refund handler)

```
async function processRefund(orderId, amount) {
  const conn = await db.getConnection();
  const order = await conn.query('SELECT * FROM orders WHERE id = ?', [orderId]);
  if (!order) throw new Error('Order not found');

  const result = await psp.refund({ orderId, amount });

  await conn.query(
    'INSERT INTO refunds (order_id, amount, psp_ref) VALUES (?, ?, ?)',
    [orderId, amount, result.refId]
  );

  const fs = require('fs');
  fs.appendFileSync('/tmp/refund-audit.log', `${orderId},${amount},${result.refId}`);

  return { success: true, refId: result.refId };
}
```

Compiles. Passes a happy-path test. Looks done.

What Forge found, running against it

4 issue(s) found before this would be safe to merge:

1. [Critical] Resource exhaustion
Connection "conn" is acquired but never released. Under sustained load this leaks connections until the pool is exhausted.
2. [Elevated] Environment drift
Writes directly to a hardcoded /tmp path. Breaks or silently loses data across containerized/autoscaled production environments.
3. [Critical] Missing defensive handling

Call to `psp.refund()` has no retry/backoff. Real networks drop requests and time out – a failure invisible until it happens in production.

4. [Critical] Unguarded irreversible action

Refund has no idempotency check. A retried request or duplicate webhook can trigger it twice – two refunds for one order.

What you'd actually review (Forge-hardened)

```
async function processRefund(orderId, amount) {
  const conn = await db.getConnection();

  try {
    const order = await conn.query('SELECT * FROM orders WHERE id = ?', [orderId])
    if (!order) throw new Error('Order not found');

    // idempotency guard – prevents duplicate refunds
    const existingRefund = await conn.query(
      'SELECT * FROM refunds WHERE order_id = ?', [orderId]
    );
    if (existingRefund) {
      return { success: true, refId: existingRefund.psp_ref, alreadyProcessed: tr
    }

    // retry/backoff – distinguishes "failed" from "response lost"
    const result = await withRetry(() => psp.refund({ orderId, amount }));

    await conn.query(
      'INSERT INTO refunds (order_id, amount, psp_ref) VALUES (?, ?, ?)',
      [orderId, amount, result.refId]
    );

    // structured logger – no hardcoded local path
    logger.info('refund_processed', { orderId, amount, refId: result.refId });

    return { success: true, refId: result.refId };
  } finally {
    // connection always released, success or failure
    conn.release();
  }
}
```

The point

You didn't review 4 problems and fix them yourself. You reviewed a diff that already had them fixed, and decided whether the fix was right — a different, faster task than finding the problem in the first place.

This is 4 mechanical categories, on one function, with hand-built pattern matching. It is not yet: aware of your specific business logic, trained on your org's incident history, or safe to trust without checking its work. That's the honest current state — and exactly what the next test needs to find out: does an engineer, shown this, want to keep checking every fix by hand, or start trusting it after the first few times it's right?